

Assignment 1

- Show how to implement a stack using two queues.
- Write an algorithm for a linear search of array $A[1:n]$ looking for element x . The algorithm should scan through the array, starting at the first element, and return the index of element x , or the special value NIL if the element is not found.
Describe a loop invariant, and use it to show your algorithm is correct.
- Design a sorting algorithm (selection sort) which follows the following procedure. First find the smallest element of $A[1:n]$ and exchange it with the element in $A[1]$. Then find the smallest element of $A[2:n]$, and exchange it with $A[2]$. Continue in this manner for the first $n-1$ elements of A .
What loop invariant does this algorithm maintain?
Give the worst-case running time of selection sort in Θ -notation. Compare with the best scenario running time.
- Find a loop invariant for the first **while** loop of the MERGE pseudocode, and use it to show the algorithm is correct.
- Describe (with pseudocode) a recursive algorithm for insertion sort, which calls on itself to sort the sub-array $A[1:n-1]$ and the inserts $A[n]$

- We saw the implementation of a queue using arrays, now implement a queue using linked lists. Is it easier to implement with or without a sentinel?
- You need to store a set of n keys in a hash table of size m . Show that if the keys are drawn from a universe U with

$$|U| > (n - 1)m,$$

then U has a subset of size n consisting of keys that all hash to the same slot. So that the worst-case searching time for hashing with chaining is $\Theta(n)$.

- What is the running time of QUICK SORT when all elements of the array have the same value?
- Banks record transactions with respect to the time of the transaction, but customers like to receive their bank statements with checks listed in order by check number. As a checkbook is ordered by check number, people tend to use their checks in order by check number. Furthermore, merchants usually cash checks reasonably promptly. Thus, the problem of converting time-of-transaction ordering to check-number ordering corresponds to the problem of sorting almost-sorted input. Explain persuasively why the INSERTION SORT algorithm might tend to beat QUICK SORT on this problem.

- An array A of size $n > 10$ is filled as follows:

For each element $A[i]$, choose two random variables x_i and y_i uniformly and independently from $[0, 1)$. Then set:

$$A[i] = \frac{\lfloor 10x_i \rfloor}{10} + \frac{y_i}{n}$$

Modify bucket sort so that it sorts the array A in $O(n)$ expected time.